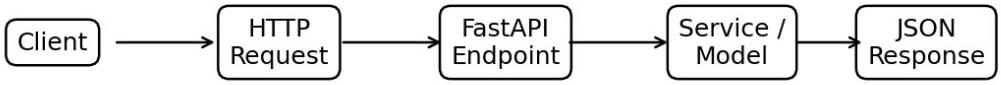
39. API DEVELOPMENT

FastAPI • REST • HTTP • JSON • Validation • Testing • Model Serving

Bir prediction API isteğinin yaşam döngüsü



REST = kaynaklar + HTTP sözleşmesi + stateless iletişim

Bu modülün amacı
Bir veri bilimi modelini veya veri servisini başka uygulamaların güvenli ve kararlı biçimde çağırabileceği bir HTTP API'ye dönüştürmek. FastAPI üzerinden request/response sözleşmesi, validation, hata yönetimi, test, authentication temelleri ve production serving mantığı öğrenilecek.

Kazanım	Modül sonunda
HTTP/REST	Method, status code, headers ve resource mantığını açıklayabilirsin.
FastAPI	Typed endpoint, path/query/body parametreleri yazabilirsin.
Validation	Pydantic ile request/response schema kurabilirsin.
Serving	ML modelini prediction endpoint'i olarak sunabilirsin.
Testing	Unit ve API integration testleri yazabilirsin.
Production	Logging, auth, timeout, versioning, health-check ve latency metriklerini tasarlayabilirsin.

1. API nedir?

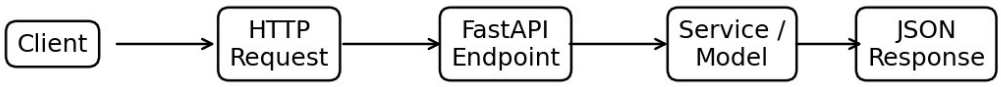
API - Application Programming Interface - iki yazılım bileşeninin hangi formatta ve hangi kurallarla iletişim kuracağını tanımlayan sözleşmedir.

Soru	API cevabı
Nereye istek atılır?	Endpoint / URL
Ne yapılacak?	HTTP method
Hangi veri gönderilecek?	Request schema
Ne dönecek?	Response schema
Hata olursa?	Status code + error body

Model serving
Bir ML modeli API arkasına konduğunda başka servisler Python kodunu bilmeden HTTP üzerinden prediction isteyebilir.

2. REST nedir?

Bir prediction API isteğinin yaşam döngüsü



REST = kaynaklar + HTTP sözleşmesi + stateless iletişim

- Resource-oriented URL
- HTTP methods
- Stateless request
- Standard status codes
- Representations - genellikle JSON

REST bir kütüphane değildir
HTTP üzerinde API tasarım yaklaşımıdır.

3. HTTP request anatomisi

Parça	Örnek
Method	POST
Path	/v1/predict
Query	?threshold=0.7
Headers	Authorization, Content-Type
Body	{"age": 42, "income": 85000}

4. HTTP methods

Method	Tipik anlam
GET	Kaynağı oku
POST	Yeni işlem/kaynak oluştur
PUT	Kaynağı tamamen güncelle
PATCH	Kısmi güncelle
DELETE	Kaynağı sil

Prediction endpoint
Tahmin işlemi body taşıdığı ve hesaplama başlattığı için çoğu durumda POST /predict mantıklıdır.

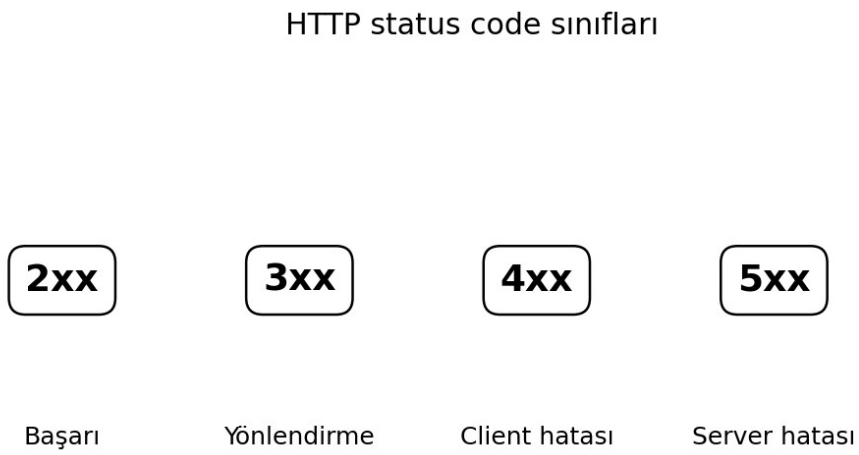
5. Idempotency ve HTTP

Aynı request birden fazla kez gönderildiğinde sistemin istenmeyen duplicate side effect üretmemesidir.

Method	Genel idempotency
GET	Evet
PUT	Genellikle evet
DELETE	Genellikle evet
POST	Garanti değil

Retry
Network timeout sonrası client aynı request'i tekrar gönderebilir; side-effect endpoint'leri idempotency key kullanabilir.

6. HTTP status codes



Code	Anlam
200	Başarılı
201	Oluşturuldu
204	Başarılı, body yok
400	Geçersiz request
401	Kimlik doğrulama gerekli
403	Yetki yok
404	Kaynak bulunamadı
409	Conflict
422	Validation error
429	Rate limit
500	Server internal error
503	Geçici olarak hazır değil

7. JSON

```
{
  "customer_id": "C123",
  "age": 42,
  "is_premium": true,
  "transactions": [120.5, 90.0]
}
```

JSON sınırı
NaN, datetime ve binary data gibi Python nesneleri doğrudan JSON değildir; serialize edilmelidir.

8. FastAPI neden?

- Python type hints tabanlı

- Pydantic validation
- Otomatik OpenAPI
- Swagger UI
- Async desteği
- Yüksek geliştirme hızı

Data science avantajı
Python model kodu ile API katmanı aynı ekosistemde kalır.

9. İlk FastAPI uygulaması

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok"}
```

Çalıştırma
Geliştirmede tipik ASGI server olarak Uvicorn kullanılır.

10. Uvicorn

```
uvicorn app:app --reload
```

Parça	Anlam
app	Python module adı
app	FastAPI object adı
--reload	Development auto-reload

Production
--reload production için değildir; process manager/container orchestration ile worker lifecycle yönetilir.

11. Path parameters

```
@app.get("/customers/{customer_id}")
def get_customer(customer_id: str):
    return {"customer_id": customer_id}
```

12. Query parameters

```
@app.get("/scores")
def scores(
    min_score: float = 0.5,
    limit: int = 100
):
    ...
```

Query param
Filtering, pagination veya optional modifiers için uygundur.

13. Request body

```
from pydantic import BaseModel

class PredictionRequest(BaseModel):
    age: int
    income: float
    tenure_months: int

@app.post("/predict")
def predict(req: PredictionRequest):
    ...
```

Typed contract
Request body otomatik parse ve validate edilir.

14. Pydantic validation

```
from pydantic import BaseModel, Field

class PredictionRequest(BaseModel):
    age: int = Field(ge=18, le=100)
    income: float = Field(ge=0)
    tenure_months: int = Field(ge=0)
```

Validation sınırı
Schema validation business rule validation'ın tamamı değildir. Örneğin müşteri hesabının aktif olup olmadığı ayrıca kontrol edilebilir.

15. Optional fields

```
class Request(BaseModel):
    customer_id: str
    campaign_id: str | None = None
```

Backward compatibility
Yeni optional field eklemek mevcut client'ları kırmadan API'yi genişletmenin yaygın yoludur.

16. Response model

```
class PredictionResponse(BaseModel):
    probability: float
    label: int
    model_version: str

@app.post(
    "/predict",
    response_model=PredictionResponse
)
def predict(req: PredictionRequest):
    ...
```

Response contract
Client'ın hangi field'ları alacağını açıkça tanımlar ve yanlışlıkla internal/sensitive alan döndürmeyi azaltır.

17. Nested schemas

```
class FeatureContribution(BaseModel):
    feature: str
    value: float

class PredictionResponse(BaseModel):
    probability: float
    explanation: list[FeatureContribution]
```

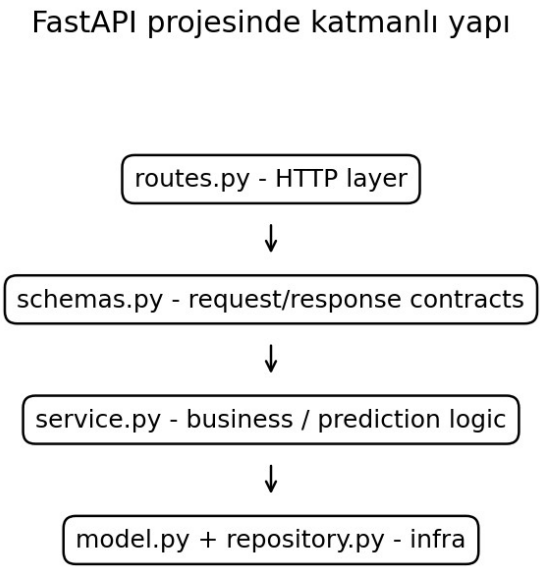
18. Enum

```
from enum import Enum

class Segment(str, Enum):
    retail = "retail"
    corporate = "corporate"
```

Fayda
Accepted values otomatik schema ve docs'a yansır.

19. Endpoint katmanını ince tut



Anti-pattern
Model loading, SQL query, preprocessing ve business rule'ların tamamını route fonksiyonunun içine yazma.

20. Service layer

```
class ChurnService:
    def __init__(self, model, transformer):
        self.model = model
        self.transformer = transformer

    def predict(self, payload):
        x = self.transformer.transform(payload)
        p = self.model.predict_proba(x)[0, 1]
        return float(p)
```

38. modül bağlantısı
Dependency injection ve katmanlı tasarım API kodunu test edilebilir hale getirir.

21. Dependency injection - FastAPI

```
from fastapi import Depends

def get_service() -> ChurnService:
    return app.state.churn_service

@app.post("/predict")
def predict(
    req: PredictionRequest,
    service: ChurnService = Depends(get_service)
):
    return service.predict(req)
```

Test avantajı
get_service dependency'sini testte override edip fake service verebilirsin.

22. Startup ve model loading

Model her request'te diskten yüklenmemelidir. Application startup sırasında bir kez yüklenip memory'de tutulur.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = load_model()
    yield
    app.state.model = None

app = FastAPI(lifespan=lifespan)
```

23. Preprocessing parity

Training'de kullanılan preprocessing inference API'de aynı artifact/pipeline üzerinden uygulanmalıdır.

Train-serving skew
Training notebook'unda ayrı, API'de elle yeniden yazılmış preprocessing büyük risk.

24. scikit-learn pipeline serving

```
pipeline = joblib.load(
    "artifacts/churn_pipeline.joblib"
)

prob = pipeline.predict_proba(
    input_df
)[:, 1]
```

En güvenli pattern
Preprocessing + model tek pipeline artifact olarak serialize edilirse parity kolaylaşır.

25. PyTorch model serving

```
model.eval()

with torch.inference_mode():
    logits = model(features)
    probability = torch.sigmoid(logits)
```

Device
Model ve tensor aynı device üzerinde olmalı; API startup'ta device kararı verilir.

26. Batch prediction endpoint

```
class BatchRequest(BaseModel):
    rows: list[PredictionRequest]

@app.post("/predict-batch")
def predict_batch(req: BatchRequest):
    ...
```

Limit koy
Binlerce satırı tek online request'e kabul etmek memory ve latency problemi yaratabilir; batch size sınırı tanımla.

27. Error handling

```
from fastapi import HTTPException

if customer is None:
    raise HTTPException(
        status_code=404,
        detail="Customer not found"
    )
```

Client vs server
Client hatasını 500 ile dönme. Doğru status code observability ve retry davranışını etkiler.

28. Custom exception handler

```
class ModelUnavailableError(Exception):
    pass

@app.exception_handler(ModelUnavailableError)
async def model_unavailable(request, exc):
    return JSONResponse(
        status_code=503,
        content={"detail": "Model unavailable"}
    )
```

29. Error response standardı

```
{
  "error": {
    "code": "CUSTOMER_NOT_FOUND",
    "message": "Customer not found",
    "request_id": "req_123"
  }
}
```

Production
Machine-readable error code, human-readable message ve request_id debugging'i kolaylaştırır.

30. Authentication vs Authorization

Kavram	Soru
Authentication	Sen kimsin?
Authorization	Bunu yapmaya yetkin var mı?

API key
Basit internal servislerde kullanılabilir; user-facing sistemlerde OAuth/OIDC/JWT gibi daha kapsamlı mekanizmalar gerekebilir.

31. API key örneği

```
from fastapi import Header, HTTPException

def verify_api_key(
    x_api_key: str = Header()
):
    if x_api_key != expected_key:
        raise HTTPException(401)
```

Secret yönetimi
expected_key source code'a yazılmaz; environment/secret manager'dan gelir.

32. JWT sezgisi

Signed token içinde subject, expiry ve claims taşınır; server token imzasını doğrular.

- sub
- exp
- aud
- roles/claims

JWT şifreli olmak zorunda değil
Payload çoğu zaman okunabilir; hassas secret token içine konmamalıdır.

33. CORS

Browser'ın bir origin'den başka origin'e API request yapmasını kontrol eden güvenlik mekanizmasıdır.

Yanlış çözüm
CORS='*' her production senaryosu için güvenli default değildir.

34. Rate limiting

Client/IP/API key başına request sayısını sınırlar.

- Abuse prevention
- Cost protection
- Fair usage

429
Limit aşıldığında HTTP 429 Too Many Requests kullanılabilir.

35. Timeout

API'nin dış dependency veya model inference için sonsuza kadar beklemesini önler.

Latency budget
Total SLA = gateway + feature lookup + preprocessing + model + postprocessing + network.

36. Retries

Hata	Retry?
Network timeout	Genellikle evet, sınırlı
503 temporary unavailable	Genellikle evet
400 validation error	Hayır
401 unauthorized	Hayır
Deterministic model bug	Hayır

Backoff + jitter
Tüm client'ların aynı anda tekrar request atmasını önlemek için.

37. Async nedir?

I/O beklerken thread'i bloklamadan başka request'lerin ilerlemesini sağlayan concurrency modelidir.

```
@app.get("/customer/{id}")
async def get_customer(id: str):
    row = await repository.fetch(id)
    return row
```

CPU-bound model
Async, ağır CPU/GPU inference'ı otomatik hızlandırmaz. CPU-bound iş worker/process veya ayrı model server gerekebilir.

38. sync vs async

Workload	Tercih
DB/network I/O	async faydalı
Kısa Python logic	sync yeterli
CPU-heavy preprocessing	Process/worker düşün
GPU inference	Batching/model server düşün

39. Concurrency ve model thread safety

Aynı model object'e eşzamanlı request'ler geliyor olabilir. Library'nin thread/process safety davranışı bilinmelidir.

Mutable state
Request sırasında shared global object'i mutate etmek race condition yaratabilir.

40. Workers

Birden çok worker process daha fazla CPU concurrency sağlar; ancak her worker modelin ayrı copy'sini memory'ye yükleyebilir.

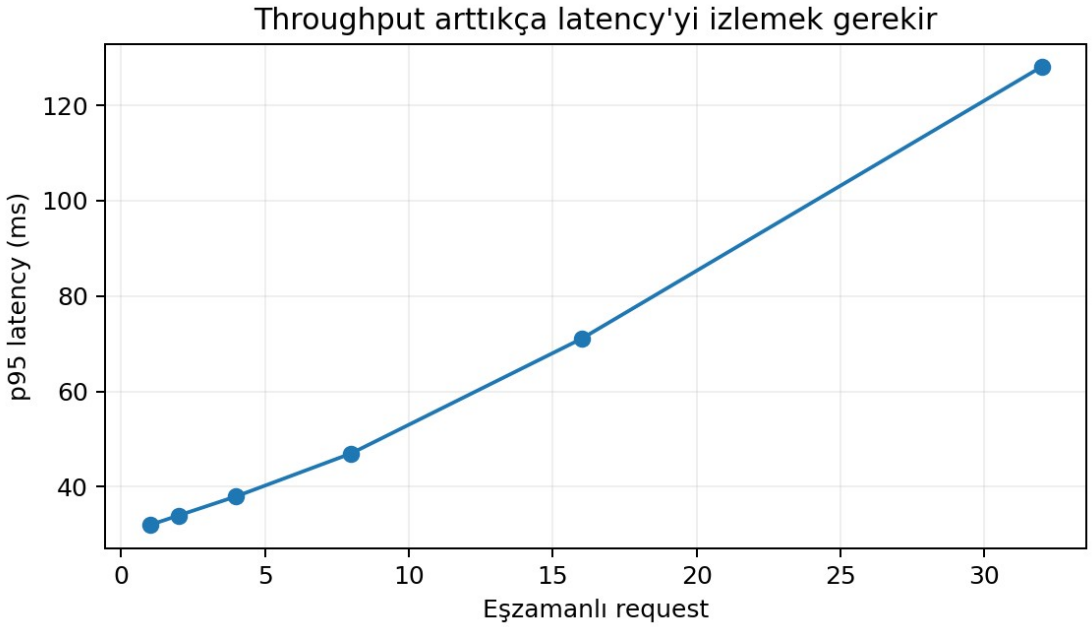
Büyük model
4 worker x 4 GB model = yaklaşık 16 GB model memory. Worker sayısı körlemesine artırılmaz.

41. Batching

Birden çok inference request'i tek model forward pass'te birleştirerek throughput artırılabilir.

Trade-off
Batch büyüdükçe throughput artabilir ama tek request'in queue latency'si de artabilir.

42. Latency metrikleri



Metric	Anlam
p50	Median latency
p95	Request'lerin %95'i bu süreden hızlı
p99	Tail latency
RPS	Request per second

Ortalama yetmez
Production kullanıcı deneyimi için p95/p99 çoğu zaman daha değerlidir.

43. Health endpoint

```
@app.get("/health/live")
def live():
    return {"status": "alive"}

@app.get("/health/ready")
def ready():
    if not model_ready():
        raise HTTPException(503)
    return {"status": "ready"}
```

Liveness vs readiness
Process yaşıyor mu? Traffic almaya hazır mı? Farklı sorular.

44. Metrics endpoint / observability

- Request count
- Error rate
- Latency histogram
- Model version
- Prediction distribution
- Feature missing rate

MLOps
API health ile model/data health aynı dashboard'da ama ayrı sinyaller olarak izlenebilir.

45. Structured logging

```
logger.info(
    "prediction_completed",
    extra={
        "request_id": request_id,
        "model_version": MODEL_VERSION,
        "latency_ms": latency_ms
    }
)
```

Loglama
Raw PII veya secret değerlerini loglama.

46. Request ID

Client -> gateway -> API -> downstream servisler boyunca taşınan correlation id.

Debug
Bir 500 hatasının tüm servislerdeki loglarını tek id ile bulabilirsin.

47. API versioning

Yöntem	Örnek
Path	/v1/predict
Header	Accept-Version: 1
Model field	model_version: v17

En basit
Public REST API için path versioning anlaşılır ve görünürdür.

48. Backward compatibility

- Existing required field'i kaldırma.
- Field type'ı sessizce değiştirme.
- Yeni field'i optional ekle.
- Deprecation dönemi tanımla.

Client contract
API schema downstream client'lar için dependency'dir.

49. Pagination

```
GET /predictions?limit=100&cursor=abc
```

Tip	Not
Offset	Basit ama büyük offset pahalı olabilir
Cursor	Stabil ve büyük dataset için iyi

50. OpenAPI ve Swagger

FastAPI type hints ve schemas üzerinden OpenAPI specification üretir; interactive docs otomatik oluşur.

Avantaj
Frontend/data consumer endpoint contract'ını kod okumadan keşfedebilir.

51. API testing

```
from fastapi.testclient import TestClient

client = TestClient(app)

def test_health():
    r = client.get("/health")
    assert r.status_code == 200
    assert r.json()["status"] == "ok"
```

52. Validation test

```
def test_invalid_age():
    r = client.post(
        "/predict",
        json={
            "age": -2,
            "income": 1000,
            "tenure_months": 4
        }
    )

    assert r.status_code == 422
```

Contract test
Invalid input'un da doğru şekilde reddedilmesi API davranışının parçasıdır.

53. Dependency override test

```
class FakeService:
    def predict(self, payload):
        return {
            "probability": 0.8,
            "label": 1,
            "model_version": "test"
        }

app.dependency_overrides[get_service] = (
    lambda: FakeService()
)
```

İyi test
API route testinde gerçek büyük modeli yüklemek zorunda değilsin.

54. Integration test

API + gerçek preprocessing + küçük test model artifact'ını birlikte çalıştır.

E2E
Container ayağa kalkıyor mu, endpoint request alıyor mu, schema doğru mu? CI'da smoke test yapılabilir.

55. Load testing

- Concurrent users
- RPS
- p95/p99 latency
- Error rate
- CPU/GPU utilization

Araç sınıfı
Locust, k6, JMeter benzeri load testing araçları kullanılabilir.

56. Capacity planning

Bir instance kaç RPS taşıyor? Hedef latency altında kaç worker/replica gerekir?

```
required_replicas
  ~= peak_rps / safe_rps_per_replica
```

Headroom
Tam kapasitede çalıştırmak yerine burst/failure için güvenlik payı bırakılır.

57. Caching

Cache	Örnek
Prediction cache	Aynı deterministic request
Feature cache	Customer feature lookup
Model cache	Loaded artifact

Cache key
Model version ve input fingerprint cache key'e dahil edilmiyorsa stale prediction dönebilir.

58. Database access

Route içinden doğrudan query yazmak yerine repository/data-access layer kullanmak maintainability sağlar.

```
class CustomerRepository:
  def get(self, customer_id: str):
    ...
```

59. N+1 query problemi

Batch request içindeki her row için ayrı DB query yapmak latency'yi patlatabilir.

Çözüm
Bulk fetch, join, caching veya feature store.

60. API gateway

- TLS

- Authentication
- Rate limiting
- Routing
- Request size limit
- Observability

Katman
Public client ile application service arasında merkezi trafik yönetimi.

61. Reverse proxy

Nginx/Envoy benzeri katman TLS termination, routing ve buffering yapabilir.

Container dünyası
Cloud load balancer veya ingress controller bu rolü üstlenebilir.

62. Deployment desenleri

Pattern	Kullanım
Rolling	Replica'ları sırayla güncelle
Blue/Green	Eski ve yeni environment yan yana
Canary	Traffic'in küçük kısmını yeni sürüme ver

ML canary
Yeni model version'ına %5 trafik verip latency ve business metric izlemek güçlü bir production pattern'dir.

63. Docker bağlantısı

```
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install .

CMD [
  "uvicorn",
  "app.main:app",
  "--host", "0.0.0.0",
  "--port", "8000"
]
```

40. modül
Bir sonraki bölümde Docker image, layer, volume, environment variable ve container deployment ayrıntılandırılacak.

64. Environment config

- MODEL_PATH
- MODEL_VERSION
- LOG_LEVEL
- DB_URL
- REQUEST_TIMEOUT

12-factor yaklaşım
Environment-specific config code'dan ayrı tutulur.

65. Graceful shutdown

Container kapanırken yeni request kabul etmeyi bırakıp mevcut request'leri tamamlamak ve resource'ları kapatmak.

Kubernetes
Readiness kaldırılır, sonra termination grace period içinde shutdown yapılır.

66. Model warm-up

İlk request'te lazy initialization/compilation nedeniyle latency spike olmasın diye startup sonrası dummy inference yapılabilir.

Özellikle
GPU, TorchScript, TensorRT veya JIT/compile kullanılan serving sistemlerinde.

67. Security checklist

1. Auth gerekiyor mu?

2. Endpoint public olmak zorunda mı?

3. Request size limit var mı?

4. Input validation var mı?

5. Secrets environment/secret manager'da mı?

6. PII loglanıyor mu?

7. Rate limit var mı?

8. Dependency vulnerabilities izleniyor mu?

68. Production checklist

1. Health/readiness endpoint var.

2. Request/response schema versioned.

3. Model startup'ta yükleniyor.

4. Structured logs + request_id var.

5. p95/p99 latency izleniyor.

6. Timeout/retry davranışı tanımlı.

7. Authentication/authorization tasarlandı.

8. Load test yapıldı.

9. Canary/rollback planı var.

10. Model/data metrics izleniyor.

69. Sık yapılan hatalar

- Modeli her request'te diskten yüklemek.
- Route içine tüm business logic'i yazmak.
- Validation olmadan raw dict kabul etmek.
- Her hatayı 500 dönmek.
- Async yazarak CPU-bound inference'ın hızlanacağını sanmak.
- Worker sayısını model memory'sini hesaba katmadan artırmak.
- PII/secrets loglamak.
- Response schema tanımlamamak.
- Health endpoint'i sadece 200 döndürmek.
- Load test yapmadan production'a çıkmak.

70. Gerçek vaka: churn prediction API

Katman	Karar
Request	customer features JSON
Validation	Pydantic
Preprocess	Serialized sklearn Pipeline
Predict	predict_proba
Response	probability + label + model_version
Observability	latency + score distribution
Fallback	503 if model unavailable

71. Gerçek vaka: fraud scoring API

İhtiyaç	Tasarım

Low latency	Online feature lookup
High RPS	Multiple replicas/batching
Strict timeout	Fail closed/open business rule
Audit	Request ID + model version
Security	Private network + service auth
Release	Canary model deployment

Business karar
Fraud modelinin timeout'ta işlemi kabul mü reddet mi edeceği teknik değil business/risk kararıdır.

72. Gerçek vaka: embedding API

Text input alıp embedding vector döndüren internal service.

- Batch input
- Max token length
- GPU batching
- Model version
- Vector dimension

Contract
Embedding model değişirse dimension/semantic space değişebilir; index ile model version birlikte yönetilir.

73. Gerçek vaka: RAG API

Adım	Endpoint içi
Validate	question + filters
Retrieve	Vector DB
Rerank	Top candidates
Generate	LLM
Response	answer + citations
Trace	retrieval ids + latency

Timeout budget
Retrieval + LLM generation toplamı online SLA içinde kalmalıdır.

74. Mülakat soruları

Soru	Kısa cevap
REST nedir?	HTTP üzerinde resource-oriented stateless API yaklaşımı.
GET vs POST?	GET read; POST create/operation ve body.
401 vs 403?	Kimlik yok/geçersiz vs kimlik var ama yetki yok.
422 ne zaman?	Request schema validation hatası.
Pydantic ne sağlar?	Typed parsing, validation ve schema.
response_model neden?	Output contract ve filtering.
modeli ne zaman yüklemeli?	Application startup/lifespan sırasında.
async ne zaman faydalı?	I/O-bound operations.

CPU-bound async olur mu?	Async tek başına CPU paralelliği sağlamaz.
p95 nedir?	Request'lerin %95'inin altında kaldığı latency.
Readiness vs liveness?	Traffic'e hazır mı vs process yaşıyor mu.
Rate limiting neden?	Abuse/cost/capacity protection.
Idempotency?	Retry sonrası duplicate side effect üretmeme.
Canary deployment?	Trafiğin küçük kısmını yeni sürüme verme.
API versioning neden?	Backward compatibility ve kontrollü değişim.

75. Uygulama alıştırmaları

1. GET /health endpoint'i yaz.

2. Pydantic ile age/income validation'lı request schema oluştur.

3. Bir sklearn pipeline'ını startup'ta yükle.

4. POST /predict endpoint'i yaz.

5. response_model ekle.

6. Invalid input için 422 test yaz.

7. Fake service ile dependency override testi yaz.

8. Request ID middleware mantığı ekle.

9. Liveness/readiness endpoint'lerini ayır.

10. Locust/k6 benzeri araçla 50 concurrent user load test planı oluştur.

76. Mini portföy projesi

Proje: Production-Ready ML Prediction API

Dosya	İçerik
app/main.py	FastAPI app + lifespan
app/routes.py	/predict + health routes
app/schemas.py	Pydantic request/response
app/service.py	Prediction workflow
app/model.py	Artifact loading
app/logging.py	Structured logging/request id
tests/	Unit + API integration tests
Dockerfile	Container image
load_test/	Load test scenario
README.md	API contract + architecture + curl examples

CV cümlesi
Built a production-ready FastAPI model-serving service with typed request/response schemas, startup model loading, dependency injection, structured error handling, health checks, automated API tests, request tracing and latency-aware deployment design.

77. Final cheat sheet

Kavram	Hatırlanacak fikir
API	Software contract
REST	Resource + HTTP semantics
GET	Read
POST	Create/operation
Status code	Machine-readable outcome
Pydantic	Validation + schema
response_model	Output contract
FastAPI Depends	Dependency injection
Lifespan	Startup/shutdown resources
Async	I/O concurrency
Worker	Process-level concurrency
p95	Tail latency
Readiness	Traffic almaya hazır mı?
AuthN	Kimsin?
AuthZ	Yetkin var mı?
Rate limit	Capacity/abuse guard
Versioning	Backward-compatible evolution
Canary	Risk-controlled release
Load test	Capacity evidence
Request ID	Distributed traceability

Bu modülden sonra
40. Docker bölümünde bu FastAPI uygulamasını image haline getirip dependency'leriyle birlikte taşınabilir bir container olarak paketleyeceksin.